Literature Notes: Software Quality Metrics for Evidence-Based Developer Reputation

Ali Bakhshayesh

February 17, 2026

How to use this file

These notes will grow paper by paper. Each paper will follow the same simple structure:

- why the paper matters for the thesis,
- what the paper says,
- how I will reuse it in thesis chapters.

Contents

1 Paper 1: *Software Metrics: A Survey* (Timoteo et al.)

1.1 Why this paper matters

This is a core reference for the thesis. It reviews software source-code metrics and explains why many proposed metrics do not survive in real use. The reasons are practical: weak validation, unclear goals, and poor tool support. Since my thesis compares metrics for developer reputation, this source helps justify focusing on metrics that are easy to interpret, stable, and difficult to misuse.

I will cite it mainly in:

- background (what software metrics are and how they evolved),
- methodology (why I select validated and interpretable metrics),
- discussion (how to explain weak or unstable metric behavior).

1.2 What the paper says

The core message is that software metrics are useful only when their meaning is clear and they are tested in realistic contexts. The paper defines metrics as quantitative indicators of software properties, but also warns that software quality is not one-dimensional and cannot be reduced to one number.

It organizes the field into two periods. The first period (before 1991) focuses mostly on complexity metrics. The second period (after 1992) shifts to object-oriented metrics, with more attention to design-level concepts such as coupling, cohesion, inheritance, and encapsulation.

For the first period, the paper discusses:

- **LOC** as a basic size indicator that is useful but highly context-dependent,
- **Cyclomatic complexity** as one of the most useful and validated metrics for control-flow complexity and testing effort,
- **Halstead metrics** as less reliable in practice because of language dependence and weaker empirical support.

For object-oriented metrics, the paper highlights the **CK suite** as the most established family due to strong validation and tool support, while also noting that no metric family is perfect. Other sets (such as Lorenz & Kidd and MOOD) are discussed as complementary but with mixed validation quality.

Another important point is automation: tools can calculate values quickly, but a number alone is not enough. Teams still need interpretation rules to connect a metric value to quality decisions.

The paper ends with a pragmatic shortlist of widely used metrics: LOC, cyclomatic complexity, CK metrics, and Lorenz & Kidd metrics.

A practical limitation is scope: this survey is centered on source-code metrics and mostly technical attributes. It does not try to model broader socio-technical drivers directly, so its metric set is better treated as a solid baseline than a complete evaluation space.

1.3 How I will reuse it in the thesis

For this thesis, it leads to four concrete choices:

1. I should compare validated metrics instead of proposing new ones.
2. Interpretability is essential: if a metric is hard to explain, it is weak for reputation.

3. Context matters: a metric can look good in one setting and fail in another.
4. Tool support helps adoption, but I still need clear rules for reading metric values.

For my comparison criteria, this paper is especially useful for:

- **stability** across versions and projects,
- **interpretability** for non-expert readers,
- **resistance to gaming** (easy-to-manipulate metrics are poor reputation signals),
- **implementation feasibility** for a small proof-of-concept smart contract.

1.4 Quick notes to remember

- Many metrics fail because goals are vague, context is ignored, or validation is weak.
- The field moved from complexity metrics to object-oriented metrics.
- The most practical survivors are LOC, cyclomatic complexity, CK, and Lorenz & Kidd.
- Automatic computation is useful, but interpretation is the real value.

1.5 Citation note

Timoteo et al. "Software Metrics: A Survey." *(venue and year to confirm from the paper PDF).*

2 Paper 2: *Measuring Software Maintainability: An Analysis of Evolving Metrics across Development Paradigms*

2.1 Why this paper matters

This is a key reference for my maintainability section. It focuses on maintainability metrics and shows how their behavior changes across development paradigms. It compares traditional, object-oriented, Agile, and DevOps contexts, and explains that metric validity is tied to context. This directly supports my evaluation goals: stability, interpretability, long-term usefulness, and resistance to misuse. Its role is mainly conceptual and discussion support, not direct definition of the experimental protocol.

I will use this paper mainly in:

- background (maintainability metrics and paradigm shifts),
- methodology (why comparison must be context-aware),
- discussion (limits of classical metrics in modern pipelines).

2.2 What the paper says

The paper starts from a practical problem: many maintainability metrics were designed for older development models and may not reflect fast iterative workflows. It asks four main questions about established metrics, continuous maintenance metrics, strengths and weaknesses, and future trends.

It frames maintainability using ISO/IEC 25010: maintainability is the degree to which software can be modified effectively and efficiently. The important point is that maintainability is multi-dimensional, so metrics are only partial proxies.

The authors run a structured literature review using major digital libraries and include mostly peer-reviewed empirical studies. They then compare findings across paradigms.

A useful contribution is the metric taxonomy:

- **Code-based metrics:** examples include cyclomatic complexity, LOC, and code churn.
- **Structure-based metrics:** examples include dependency and information-flow metrics (fan-in/fan-out style reasoning).
- **Hybrid metrics:** examples include the Maintainability Index and CK metrics.

The paper notes a tradeoff in hybrid metrics: they are convenient for summary, but harder to interpret when you need a clear cause behind a score.

The paper also discusses continuous practices in Agile and DevOps, where maintenance is not a separate late phase but part of daily development. In this context, teams often track operational metrics like MTTF, MTTR, deployment frequency, and change failure rate.

The key caution is that process and operational metrics do not directly measure code maintainability. They are useful, but they should be interpreted together with code and structure metrics. They can also be strongly affected by tooling choices and team process, so they are easier to optimize without necessarily improving code quality.

The final message is that no single metric is enough for all contexts. Metric choice should be tied to paradigm, system type, development goals, and newer settings such as microservices.

One limitation to keep in mind is evidence heterogeneity: the paper aggregates results from different study designs and contexts, but there is no single unified benchmark across all paradigms. So cross-paradigm conclusions are strong conceptually, but still need local empirical validation.

2.3 How I will reuse it in the thesis

From this paper, I derive four practical decisions:

1. I should test metric behavior across contexts, not assume one-size-fits-all validity.
2. I should favor metrics with clear meaning over opaque composite indices, and avoid blind use of fixed thresholds.
3. I should treat process metrics as complementary signals, not direct code-quality proxies or primary experiment metrics.
4. I should discuss gaming risk for metrics connected to team incentives (for example, deployment frequency alone).

For my comparison framework, this paper is especially useful for:

- **stability** across versions and paradigms,
- **interpretability** of single metrics versus aggregated indices,
- **long-term suitability** under evolving development practices,
- **resistance to gaming** when metrics become performance targets.

2.4 Quick notes to remember

- Maintainability cannot be captured by one metric.
- Maintainability metrics are paradigm-dependent.
- Code, structure, and hybrid metrics each capture different dimensions.
- Fixed thresholds can be misleading in fast-changing systems.
- DevOps metrics mainly reflect delivery and operations, not code quality alone.
- Context-aware metric selection is essential.

2.5 Citation note

Syed-Mohamad et al. "Measuring Software Maintainability: An Analysis of Evolving Metrics across Development Paradigms." *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 2026.

3 Paper 3: *A Survey of Key Factors Affecting Software Maintainability*

3.1 Why this paper matters

I use this paper mainly for conceptual grounding. It does not compare metrics experimentally, but it gives a strong foundation for definitions and terminology. It helps separate maintainability factors from maintainability metrics, and it anchors the discussion in widely used standards.

I will use this paper mainly in:

- background (definition of maintainability and maintenance),
- methodology (mapping metrics to factors),
- discussion (limits of metric-only interpretations).

3.2 What the paper says

The paper starts from a practical motivation: maintenance can consume around 65% to 75% of total lifecycle effort, so maintainability has a major cost impact.

It clearly distinguishes two terms:

- **Maintainability:** a property of the software system.
- **Maintenance:** the activity of modifying software after delivery.

This distinction matters because maintenance effort alone is not a direct measure of maintainability.

The paper reviews the four classic maintenance types: corrective, adaptive, perfective, and preventive. It also uses quality-model references (especially ISO/IEC 9126) to show that maintainability is a high-level attribute with sub-characteristics such as analysability, modifiability, stability, and testability.

Another key point is that many models exist (including ISO-based, Boehm/McCall style, and fuzzy-logic approaches), but no single model fully captures maintainability in all contexts.

The core contribution is a list of factors that affect maintainability:

- **Code factors:** size, complexity, coupling, cohesion, inheritance.
- **Design factors:** modularity, architecture quality, component structure.
- **Readability/documentation factors:** comments, naming, clarity.
- **Testing factors:** testability and test coverage maturity.
- **Human and organizational factors:** developer skills, tool support, turnover.

The paper emphasizes that metrics are indicators of these factors, not direct measurements of maintainability itself.

The evidence style is mostly conceptual synthesis: this work consolidates standards and prior models, but it is not an empirical benchmark paper comparing large datasets across tools or ecosystems.

3.3 How I will reuse it in the thesis

For my thesis design, this paper leads to four decisions:

1. I should treat maintainability as a latent, multi-factor property.
2. I should explicitly map each selected metric to a factor, instead of claiming it measures maintainability directly.
3. I should use standards-based vocabulary to keep definitions defensible.
4. I should acknowledge human and organizational effects when interpreting metric results.

For my comparison framework, this paper especially supports:

- **interpretability** through clear factor-to-metric mapping,
- **stability** as a formal maintainability sub-characteristic,
- **long-term suitability** through design and documentation factors,
- **resistance to gaming** by warning that a metric can improve while real maintainability does not.

3.4 Quick notes to remember

- Maintainability is a software property; maintenance is an activity.
- Maintainability cannot be measured by one metric.
- Metrics approximate contributing factors, not the full attribute.
- ISO/IEC 9126 gives a solid structure for maintainability reasoning.
- Human and organizational factors matter, even when not directly measured.

3.5 Citation note

Balraj Kumar "A Survey of Key Factors Affecting Software Maintainability." *IEEE International Conference on Computing Sciences (ICCS)*, 2012.

4 Paper 4: *An Empirical Analysis of the Maintainability Evolution of Open Source Systems*

4.1 Why this paper matters

- **Verdict:** Core paper (empirical and methodological backbone).
- **Role in the thesis:** This paper provides direct empirical evidence on how maintainability metrics behave across versions of real open-source systems and serves as a methodological reference for studying metric stability over time.
- **Primary contribution to the thesis:** It demonstrates that maintainability metrics evolve non-linearly, that correlations between metrics are project-dependent, and that independent empirical validation is necessary.

4.2 What the paper says

Motivation and problem statement The paper starts from a critical observation: although many software maintainability models and metrics have been proposed in the literature, they are rarely evaluated independently or reused across different studies. Most models are validated only by their proposers, often on a limited number of projects, making it difficult to assess their general applicability. The authors propose a different approach, focusing on the independent application of existing maintainability metrics to popular open-source systems in order to analyze their evolution over time.

Maintainability as an evolving property Maintainability is treated as a dynamic quality attribute that changes as software evolves. As systems grow, refactor, and adapt to new requirements, their size, complexity, and structure change, which in turn affects maintainability. This motivates the need to analyze maintainability across multiple versions rather than at a single snapshot in time.

Selected metrics The study focuses on three widely used metrics:

- **Cyclomatic Complexity (CC):** CC measures the number of independent execution paths through the source code. Higher values indicate greater structural complexity and are generally associated with increased maintenance effort.
- **Lines of Code (LOC):** LOC is used as a simple measure of software size. While easy to compute and interpret, its relationship with maintainability is context-dependent.
- **Maintainability Index (MI):** MI is a composite metric combining Halstead Volume, cyclomatic complexity, and LOC. It provides a single numerical indicator intended to summarize maintainability, with commonly used threshold ranges (for example difficult, maintainable, and good).

Case study design The authors analyze multiple open-source Java projects, starting with a detailed case study of the jEdit project. Twelve major versions of jEdit are examined, covering a long period of its evolution. Metrics are extracted using the JHawk static analysis tool. Both variants of the MI metric (with and without comments) are discussed, and the authors justify their choice of including comments despite the lack of standardization.

Empirical results The analysis reveals that MI does not evolve monotonically. In jEdit, MI remains stable across several versions, drops sharply after major changes, and later recovers. Cyclomatic complexity tends to be higher in versions with lower MI, while LOC shows an inconsistent relationship with MI. Similar analyses on four additional projects (SoundHelix, jWebUnit, jXLS, jTDS) show heterogeneous behaviors, with significant correlations appearing only in some projects.

Statistical analysis To assess relationships between metrics, the authors use Spearman rank correlation, chosen because it does not assume linear relationships or specific data distributions. Results indicate that correlations between LOC, CC, and MI vary across projects, and in some cases are weak or non-significant. This suggests that the behavior of composite metrics such as MI is highly project-specific.

Limitations The authors explicitly acknowledge several limitations, including the restricted number of analyzed projects, language dependence (Java), and reliance on a single static analysis tool. These limitations are presented transparently and motivate future research rather than undermining the study's conclusions.

4.3 How I will reuse it in the thesis

Key insights for the thesis

1. **Metric stability must be measured empirically.** This paper shows that maintainability metrics can fluctuate significantly across versions, even within the same project.
2. **Composite metrics are risky.** MI aggregates multiple dimensions and may hide the underlying causes of maintainability changes, making it less suitable for transparent reputation assessment.
3. **Metric behavior is project-dependent.** There is no universal relationship between size, complexity, and maintainability across all systems.
4. **Independent validation is essential.** Applying existing metrics to new datasets provides more insight than proposing new metrics without validation.

Support for thesis comparison criteria For the thesis criteria, this paper gives strong support on the following dimensions:

- **Stability:** Non-linear evolution and abrupt changes in MI highlight the need to quantify stability across versions.
- **Interpretability:** Simple metrics such as CC are easier to reason about than composite indices.
- **Long-term suitability:** Metrics that behave inconsistently across projects may be unsuitable as long-term reputation indicators.
- **Resistance to gaming:** Composite metrics can potentially be manipulated by optimizing subcomponents without improving overall code quality.

Methodological reuse The study provides a concrete methodological template that can be reused in this thesis, including:

- multi-version analysis of open-source projects,
- use of widely available static analysis tools,
- correlation-based investigation of metric relationships,
- cautious interpretation of results.

Where this paper will be used

- **Chapter 2:** Empirical background on maintainability evolution.
- **Chapter 3:** Inspiration and justification for experimental design choices.
- **Chapter 4:** Interpretation of metric stability, inconsistencies, and limitations.

4.4 Quick notes to remember

- Maintainability evolves non-linearly over time.
- Metric correlations are not stable across projects.
- Composite metrics may obscure causal relationships.
- Empirical, independent evaluation is necessary.

4.5 Citation note

Kapllani, G., Khomyakov, I., Mirgalimova, R., Sillitti, A. "An Empirical Analysis of the Maintainability Evolution of Open Source Systems." *IFIP AICT 582, OSS 2020.*

5 Paper 5: *Exploring Factors and Metrics to Select Open Source Software Components for Integration: An Empirical Study*

5.1 Why this paper matters

- **Verdict:** Core support paper (feasibility, availability, and proxy-metric risks).
- **Role in the thesis:** Although the paper targets OSS component selection (not developer reputation), it provides strong empirical evidence about (i) how practitioners operationalize quality factors via metrics, and (ii) which metrics are realistically extractable at scale from OSS portals/APIs. This directly supports the thesis requirement to include computability/feasibility (including smart-contract feasibility) as a metric selection criterion.
- **Primary contribution to the thesis:** The paper demonstrates that many desirable quality signals are not consistently available as machine-readable metrics; proxy metrics are common but introduce interpretability and gaming risks.

5.2 What the paper says

Motivation and scope Open-source components are widely integrated into industrial software products, but selecting an OSS component is a multi-criteria decision with practical constraints. The paper investigates which factors practitioners consider when selecting OSS components for integration, which metrics they use to operationalize those factors, and to what extent these metrics can be automatically extracted from public OSS portals. The core perspective is practitioner-centered: selection criteria should reflect real-world decision-making and data availability.

Research questions and study design The study is organized around four main questions: (i) what factors and metrics practitioners use to evaluate OSS components, (ii) what information sources practitioners rely on, (iii) what factors/metrics can be assessed automatically from OSS portals, and (iv) what improvements would increase the usefulness of OSS portals for selection decisions. The authors employ a two-phase approach:

- **Qualitative elicitation:** semi-structured interviews with 23 experienced practitioners to identify factors and metrics used in OSS selection.
- **Large-scale validation:** an API-based analysis of the most starred 100,000 GitHub repositories to determine which metrics are actually available and consistently populated at scale.

Factors versus metrics (conceptual separation) A key conceptual distinction is made between:

- **Factors:** high-level qualities decision makers care about (e.g., community support, maturity, quality, risk, trustworthiness).
- **Metrics:** operational measurements used to approximate factors (e.g., number of contributors, release frequency, issue resolution time).

The paper emphasizes that many factors do not have direct metrics and are often approximated by one or more proxy metrics. This creates an interpretation gap between the numeric signal and the underlying concept.

Identified factors and metric inventory From practitioner input, the authors identify eight high-level factors (each decomposed into multiple sub-factors). They consolidate and extend the metric inventory using prior literature, resulting in a large set of metrics associated with OSS selection. A central observation is that selection decisions commonly combine multiple dimensions (technical quality, community activity, adoption, legal constraints), rather than relying on a single measure.

Automatability and data availability at scale The most thesis-relevant contribution is the systematic assessment of metric availability via portal APIs. Despite the large metric inventory (reported as 170 metrics overall), the paper finds that only a minority are realistically automatable:

- Only **40** metrics are potentially available through APIs.
- Only **22** metrics return information consistently across the full dataset of 100,000 repositories.

This implies that many theoretically relevant criteria cannot be operationalized as automated evidence in a robust, cross-project manner. The paper therefore recommends improving OSS portals by providing more standardized metadata and richer machine-readable indicators.

Interpretation risks and limitations The paper highlights that practitioners frequently use proxy metrics (e.g., stars, commits, issue activity) because they are available. However, these metrics are context-dependent and may capture popularity or activity rather than intrinsic code quality. The authors discuss that selection must therefore balance automatable evidence with contextual interpretation and sometimes manual inspection.

5.3 How I will reuse it in the thesis

Why this paper is strategically important for the thesis

1. **Feasibility is a first-class selection criterion.** The study provides empirical justification for including availability/computability as a core criterion in metric selection for evidence-based reputation systems.
2. **Factors $\neq$ metrics.** The factor/metric separation is directly reusable to argue that reputation cannot be reduced to a single raw metric, and that metrics are proxies for underlying quality dimensions.
3. **Proxy metrics are unavoidable but risky.** Many desirable qualities (e.g., trustworthiness) lack direct measures; available proxies can be ambiguous, confounded, or manipulable.

Support for thesis comparison criteria For my comparison framework, this paper adds strong support on these dimensions:

- **Stability:** Portal-based activity metrics can fluctuate due to attention, trends, or external events; this supports the need to test stability empirically rather than assume it.
- **Interpretability:** Many available metrics (stars, forks, commits) are easy to understand but ambiguous in meaning; this supports evaluating interpretability as human explainability rather than mere simplicity.
- **Resistance to gaming:** Counts and activity-based measures are susceptible to manipulation (e.g., artificial stars, superficial commits); this supports treating gaming resistance as a key selection criterion.

- **Long-term suitability:** Metrics tied to platform popularity may not reflect long-term developer quality; metrics with stable semantic meaning across time are preferable.
- **Smart-contract feasibility:** Metrics that are not consistently obtainable or reproducible are poor candidates for on-chain proof-of-concept implementation; this paper provides evidence for that constraint.

How to reuse this paper in thesis writing (concrete guidance)

- **Chapter 3 (Methodology / criteria):** cite this paper to justify adding availability and computability as a metric selection criterion alongside conceptual criteria (stability, interpretability, gaming resistance).
- **Chapter 4 (Discussion):** use it to explain why certain intuitive indicators are excluded or down-weighted (data missingness, confounding).
- **Threats to validity / limitations:** use the large-scale API findings to argue that metric-based systems may be biased toward projects that expose rich metadata and may under-represent high-quality but low-visibility projects.

5.4 Quick notes to remember

- Practitioners evaluate OSS with multiple factors; metrics are proxies.
- Many metrics are not available or not consistently populated across projects.
- Only a minority of metrics are automatable at scale (40 available; 22 consistently available across 100k repos).
- Proxy metrics introduce interpretability and gaming risks.

5.5 Citation note

Li, Moreschini, Zhang, Taibi. "Exploring Factors and Metrics to Select Open Source Software Components for Integration: An Empirical Study." *(venue/journal and year to confirm from the paper PDF).*

6 Paper 6: *Towards Evidence-Based Software Quality Practices for Reproducibility: Practices and Aligned Software Qualities*

6.1 Why this paper matters

- **Verdict:** Strong support paper (evidence-based framing and ISO/IEC 25010 alignment).
- **Role in the thesis:** This paper strengthens the philosophical and methodological framing of the thesis: it provides empirical evidence about which software engineering practices support reproducibility and which ISO/IEC 25010 product quality characteristics are perceived to align with reproducibility. Although the paper is situated in computational science and engineering (CSE), its findings are useful to motivate maintainability and other quality attributes as defensible, evidence-based targets for metric-driven assessment.
- **Primary contribution to the thesis:** It empirically links reproducibility to maintainability (and reliability/usability) and highlights auditable practices (testing, documentation, version control) as central for evidence-based quality.

6.2 What the paper says

Motivation and scope The paper argues that reproducibility is a foundational requirement in computational science and engineering (CSE), but that teams face practical constraints and must prioritize among competing software qualities. Despite the importance of reproducibility, there is limited empirical evidence about (i) what development practices facilitate reproducibility, and (ii) which product quality characteristics are most aligned with reproducibility. The authors therefore adopt an evidence-based approach to identify practices and quality attributes that are perceived as relevant for reproducible software.

Definitions and standards framing Reproducibility is defined according to NASEM (2019) as obtaining consistent results using the same input data, computational steps, methods, code, and analysis conditions. The paper uses ISO/IEC 25010 as the product quality framework and explicitly notes that reproducibility is not itself a standard product quality characteristic, motivating the need to study how it relates to existing quality characteristics.

Research questions The study is driven by three questions:

- **RQ1:** What development practices facilitate reproducible software?
- **RQ2:** What do developers believe about the relationship between software quality and reproducibility, and are they willing/able to improve software quality for reproducibility?
- **RQ3:** Which ISO/IEC 25010 product quality characteristics are perceived to contribute to reproducibility, and how do projects prioritize them?

Methodology The authors use a mixed-method design:

- **Case studies:** They conduct case studies of nine CSE software teams at Sandia National Laboratories. Interviews are analyzed using qualitative coding and thematic grouping (inspired by Grounded Theory) to identify categories of practices used to support reproducibility.
- **Survey:** They conduct an online survey with 219 CSE software developers and users (both Sandia and non-Sandia). The survey includes Likert items about reproducibility beliefs and willingness, ranking of ISO/IEC 25010 characteristics, and a MaxDiff (best-worst) exercise to estimate which qualities contribute most/least to reproducibility. Rankings are analyzed using Borda counts; MaxDiff choices are analyzed via multinomial logit modeling with regularization. They also apply corrections for multiple comparisons where appropriate.

Findings for RQ1: practices supporting reproducibility From the case studies, seven categories of practices are identified:

- automated testing,
- code reviews,
- development pipeline,
- documentation,
- public processes,
- team communication,
- version control.

Three practices were universally highlighted across all nine teams: automated testing, documentation, and version control. The paper argues these practices produce artifacts that directly support reproducibility by making expected behavior explicit (tests), clarifying intended use and constraints (documentation), and enabling traceability and rollback (version control).

Findings for RQ2: beliefs and willingness Survey respondents overwhelmingly view reproducibility as essential and believe that higher-quality software is more likely to be reproducible. Many respondents express willingness to invest additional effort to improve software quality for reproducibility. However, a substantial fraction report limited ability to identify and address reproducibility issues quickly and request more training and support. Respondents also indicate varying time horizons for reproducibility (for example months versus years/decades), illustrating realistic constraints and differing expectations in practice.

Findings for RQ3: aligned product qualities Using the MaxDiff analysis, respondents perceive reproducibility as most aligned with maintainability, reliability, and usability (followed by functional suitability). In contrast, security and performance efficiency are perceived as least aligned (and sometimes negatively aligned) with reproducibility in the studied context. The authors further explore how project priorities influence perceived utility and identify respondent clusters based on quality prioritization, emphasizing that teams may adopt different quality profiles.

Discussion themes and limitations The paper discusses several cross-cutting themes: communicating intended use, learning from others, involving software engineering expertise, and right-sizing solutions to project context. Reported barriers include lack of training, limited institutional support, and diverse definitions of reproducibility across communities. Threats to validity include contextual bias toward national lab environments and the possibility that unreported practices still exist in teams.

6.3 How I will reuse it in the thesis

Strategic value for this thesis I use this paper mainly as an evidence-based framing reference, not as a direct metric-comparison study:

1. **Evidence-based quality mindset:** The paper provides a strong external anchor for arguing that quality practices and quality targets should be justified with empirical evidence rather than convention.

2. **Maintainability as a defensible target:** The strong perceived alignment between reproducibility and maintainability supports treating maintainability as a central long-term quality dimension, which strengthens the rationale for maintainability-related metrics as reputation-relevant signals.

3. **Auditable practice artifacts:** Testing, documentation, and version control are highlighted as universally important practices, and they produce artifacts that can be audited and measured, supporting the evidence perspective of reputation systems.

Mapping to thesis comparison criteria

- **Stability:** The emphasis on version control and testing supports the importance of tracking changes across versions and motivates stability analysis as a meaningful property.
- **Interpretability:** Maintainability, reliability, and usability are understood and prioritized by practitioners, supporting interpretability as a key criterion for reputation metrics.
- **Resistance to gaming:** Practice-based evidence can be gamed superficially (e.g., minimal documentation), motivating explicit discussion of gaming resistance for any evidence-based metric system.
- **Long-term suitability:** The paper's discussion on time horizons (months/years/decades) provides language for why long-term indicators matter and why objectives differ by context.
- **Feasibility:** While not focused on smart contracts, the paper reinforces that measurable, auditable evidence often comes from artifacts (VCS history, tests, docs), which tends to be more feasible for automation than subjective qualities.

How to reuse this paper in writing

- **Chapter 2 (Background):** Use it to motivate evidence-based quality practices and to connect reproducibility to ISO/IEC 25010 qualities.
- **Chapter 3 (Criteria definition):** Cite it when motivating maintainability as a central, defensible target quality and when explaining why interpretability matters to stakeholders.
- **Threats to validity / limitations:** Use its limitation structure as a template for discussing context specificity and sampling bias.

6.4 Quick notes to remember

- Universally emphasized practices for reproducibility: automated testing, documentation, version control.
- Reproducibility perceived to align most with maintainability (then reliability/usability).
- Evidence-based quality requires context-aware prioritization and trade-off awareness.

6.5 Citation note

Gonzales et al. "Towards Evidence-Based Software Quality Practices for Reproducibility: Practices and Aligned Software Qualities." *ACM REP '24, 2024.*

7 Paper 7: *Reputation Systems Evaluation: A Survey*

7.1 Why this paper matters

- **Verdict:** Highly aligned (core support paper for the reputation-systems side).
- **Role in the thesis:** This survey provides the missing other half of the thesis framing: while several papers in this review focus on software quality metrics, this paper focuses on how reputation systems themselves are evaluated. It supports defining a defensible, explicit evaluation protocol for metric-driven reputation signals (stability, interpretability, resistance to gaming, long-term suitability) and motivates threats/attack scenarios as standard evaluation dimensions.
- **Primary contribution to the thesis:** It provides a taxonomy of evaluation approaches (custom experiments, restricted common experiments, common evaluation frameworks) and a consolidated view of evaluation metrics, threat models, and desiderata for reproducible benchmarking.

7.2 What the paper says

Motivation Reputation systems are used to estimate the trustworthiness of entities and support decision making in distributed environments. Because different reputation systems make different architectural and modeling assumptions, and because experimental setups are often custom-made, it is difficult to compare systems objectively across the literature. The paper therefore surveys how reputation systems are evaluated and argues that the field lacks standardized evaluation methodologies.

Taxonomy of evaluation approaches The authors propose a taxonomy of reputation system evaluation approaches:

- **Custom-made experiments:** Most research evaluates a new reputation system using bespoke simulations and metrics. This creates comparability problems because definitions of evaluation metrics and threat scenarios vary widely.
- **Common experiments under restricted scenarios:** Some studies reuse shared scenarios (e.g., e-market benchmarks, Prisoner's Dilemma/utility games), but these are limited in scope and are not generic.
- **Common evaluation frameworks (CEFs):** Frameworks that aim to standardize evaluation and enable reproducible comparison across systems.

CEFs are further divided into theoretic and simulation-based frameworks, reflecting whether evaluation is based on criteria/threat analysis or on shared simulation/testbed tooling.

Evaluation metrics and what is measured The paper identifies two major families of evaluation metrics:

- **Correctness / prediction quality:** metrics that measure how accurately a system identifies trustworthy vs untrustworthy entities or predicts future behavior (e.g., success rates, detection rates, false positives/negatives, RMSE-like measures depending on the model).
- **Performance / overhead:** metrics that measure computational and communication costs (e.g., convergence time, message overhead, storage/runtime cost).

A recurring message is that even when the same term is used (for example convergence), different works operationalize it differently, which undermines cross-paper comparison.

Threat models and attack scenarios The survey reviews commonly simulated attacks used to evaluate robustness of reputation systems, including:

- **Badmouthing** (unfairly negative recommendations),
- **Collusion** (groups boosting each other or defaming others),
- **Oscillating behavior** (alternating honest and dishonest behavior),
- **Traitor attacks** (behave well until high reputation, then exploit it).

The paper stresses that attack modeling differs widely across studies and that evaluation outcomes depend strongly on assumptions about dynamic behavior (nodes joining/leaving, mobility, topology changes) and social relationships.

Common evaluation frameworks and their limitations The survey synthesizes why CEFs have not achieved broad adoption:

- frameworks often target a specific architecture (centralized vs decentralized) or a specific environment (e-commerce, P2P, WSN),
- limited coverage of threat models and interaction patterns,
- inconsistent definition and reporting of evaluation metrics,
- limited extensibility and scarce availability of reusable implementations.

A comparative view of CEF proposals is provided along dimensions such as standardization, independence from system characteristics, flexibility, ease of implementation, and availability of implementations.

Desiderata for a widely useful evaluation framework The paper proposes key requirements for a widely accepted CEF:

- **Standardization** (common abstractions and shared evaluation metrics),
- **Independence from system characteristics** (supporting diverse reputation mechanisms),
- **Flexibility** (configurable environments, threats, social relationships, dynamic behaviors),
- **Ease of adding new systems/tests**,
- **Availability of implementations** (to enable reproducibility).

The authors also discuss practical factors that enable these requirements, such as abstract system interfaces, standardized time/interaction models, configurable attacker profiles, and balancing flexibility with comparability.

7.3 How I will reuse it in the thesis

Why this paper is foundational for the reputation half of the thesis In my thesis, I link software quality metrics to developer reputation. This paper provides the vocabulary and justification for treating evaluation methodology and robustness to attacks/gaming as essential, not optional. It helps justify explicit comparison criteria and keeps gaming resistance as a legitimate evaluation axis.

Reusable arguments for thesis writing

Argument A: explicit evaluation criteria are required Because reputation evaluations are often non-standardized and not comparable, the thesis must define a clear evaluation protocol for metric selection and validation (cross-version, cross-project, and feasibility checks). This motivates presenting criteria (stability, interpretability, resistance to gaming, long-term suitability) as a structured evaluation design rather than subjective preferences.

Argument B: threat modeling is part of reputation evaluation The survey shows that robustness against attacks (badmouthing, collusion, oscillation, traitor) is standard in reputation system evaluation. Metric gaming analysis can be framed as an analog of these attacks in a developer-quality context (e.g., collusion-like mutual boosting, oscillation-like bursty good activity, traitor-like long honest history followed by harmful changes).

Argument C: a lightweight CEF-inspired evaluation is acceptable for minimum thesis scope The thesis is not building a universal CEF, but it can design a small, defensible evaluation protocol inspired by CEF desiderata:

- standardized data pipeline and metric computation,
- explicit scenarios: cross-project and cross-version evaluation,
- stability measures defined operationally,
- basic robustness/sensitivity checks (lightweight gaming tests),
- reproducible reporting of metrics and results.

How this paper supports comparison criteria

- **Stability:** parallels the idea of convergence and dynamic behavior; stability must be operationally defined to be comparable.
- **Interpretability:** standardization and consistent definitions are required to make results meaningful.
- **Resistance to gaming:** aligns with attack scenarios as a primary evaluation dimension.
- **Long-term suitability:** relates to time modeling, recency, and dynamic participants.
- **Feasibility / smart-contract computability:** reproducible evaluation and standardized data availability motivate selecting deterministic, auditable metrics for on-chain proof-of-concept.

Where to use it in the thesis

- **Chapter 2:** reputation systems background plus evaluation challenges and taxonomy.
- **Chapter 3:** justification for the evaluation protocol and for including gaming resistance and reproducibility as dimensions.
- **Threats to validity / limitations:** use the survey's comparability critique and threat-model variability as a template for limitation discussion.

7.4 Quick notes to remember

- Reputation system evaluation lacks standardization; custom experiments dominate.
- Common evaluation frameworks aim to standardize but are limited; reproducibility and availability of implementations matter.
- Correctness vs overhead are the main families of evaluation metrics.
- Robustness to attack scenarios (badmouthing, collusion, oscillation, traitor) is a standard evaluation axis.

7.5 Citation note

Hassan, A., et al. "Reputation Systems Evaluation: A Survey." *ACM Computing Surveys (CSUR), 2015/2016 (confirm exact year/volume from the paper PDF).*

8 Paper 8: *Comparing and Evaluating Metrics for Reputation Systems by Simulation*

8.1 Why this paper matters

- **Verdict:** Core paper (evaluation framework and robustness for reputation metrics).
- **Role in the thesis:** This paper provides a concrete foundation for evaluating reputation system metrics by simulation. It strengthens the methodology of this thesis by illustrating how different reputation metrics perform in adversarial settings, making it highly relevant for gaming resistance and the evaluation framework.
- **Primary contribution to the thesis:** It demonstrates how to simulate and evaluate reputation metrics under common attacks and adversarial behavior, offering both a formal model and empirical findings regarding metric robustness.

8.2 What the paper says

Motivation and problem framing Reputation systems, used to assess the trustworthiness of agents in distributed systems, require effective evaluation metrics to function in hostile or unreliable environments. The paper aims to address the lack of comparability between reputation systems by using a formal model for reputation metrics and evaluating them through simulation experiments under various adversarial conditions.

The authors provide a structured simulation framework and apply it to evaluate several widely known reputation metrics, including EigenTrust, OnlyLast, Accumulative, and Blurred systems, comparing them under different attack models, such as malicious raters, collusion, and traitor attacks.

Formal model for reputation systems The authors define a formal model for reputation metrics, structuring reputation as a combination of a rating function and a reputation computation function. Agents rate each other after transactions, and reputation is calculated as a function of past ratings (potentially weighted by time, transaction value, and history). The paper formalizes this setup using an agent-based simulation framework (RePast) where different reputation metrics can be evaluated consistently.

This formal model is particularly useful for this thesis because it offers a clear, reusable framework for structuring and evaluating reputation systems, which can be adapted to evaluate software quality metrics as part of developer reputation systems.

Simulated reputation metrics The paper evaluates multiple reputation metrics, including:

Accumulative reputation systems These sum all ratings without time decay. This is the most straightforward approach, but it is vulnerable to manipulation by high-frequency raters or attacks like badmouthing.

Average-based metrics These calculate the average rating. They offer more robustness than simple accumulative methods but still suffer from collusion and badmouthing.

Time-decayed metrics (Blurred) The Blurred metric decays older ratings over time. This approach is more resistant to manipulative actions over long periods and captures recent activity as more significant than old ratings.

OnlyLast metric Only the last rating counts. While simple and highly resistant to some forms of manipulation, this metric fails to handle oscillating behaviors or periodic malicious activities effectively.

EigenTrust This is a more sophisticated approach using iterative updates to compute a global trust value. EigenTrust is effective against collusion, but it can fail in systems where agents are not sufficiently connected, or when ratings are sparse.

BlurredSquared (proposed metric) The authors propose BlurredSquared, a modification of Blurred that gives quadratic weight to recent ratings, improving robustness against oscillating behaviors and collusion. This new metric shows promise for robustness against common adversarial strategies.

Adversarial behavior and attack scenarios The simulation includes multiple attack scenarios to assess the robustness of each metric:

Badmouthing Malicious agents provide negative ratings to damage other agents' reputations, even when they do not engage in fraudulent behavior themselves.

Collusion Agents group together and coordinate positive or negative ratings to boost or degrade each other's reputation unfairly.

Traitors (oscillating behavior) Agents initially behave honestly to build reputation but later exploit the trust they have gained to behave maliciously.

Key results

- OnlyLast is surprisingly effective against traitor behavior but weak against badmouthing and collusion.
- EigenTrust is robust to collusion but vulnerable where some agents are free riders or provide too little feedback.
- Blurred and BlurredSquared are robust across several attack models, but BlurredSquared shows superior resistance against oscillating agents. However, all models show limitations under certain attack scenarios.
- Accumulative and Average models struggle significantly under malicious raters and badmouthing.
- The authors argue that a single metric may not be universally effective and advocate multi-metric evaluation.

Limitations The authors explicitly acknowledge limitations:

- Their simulation environment was not always reflective of real-world complexities (for example the number of agents and types of interactions).
- They did not account for every possible form of attack, and results are specific to the modeled agent and attack patterns.
- There is an implied trade-off between metric complexity and robustness: simpler metrics like OnlyLast are more transparent but less resistant to sophisticated attacks.

8.3 How I will reuse it in the thesis

Key insights for the thesis

1. **Gaming resistance is critical.** The paper demonstrates that different reputation metrics are vulnerable to different attack types, supporting gaming resistance as a critical comparison dimension.
2. **Metric robustness varies under adversarial conditions.** No single metric works under all attacks. This justifies a multi-metric approach and the need to test stability under different scenarios.
3. **Time-decayed metrics show improved robustness.** The focus on time-decayed ratings (BlurredSquared) and the benefits of quadratic weighting aligns with the thesis emphasis on long-term stability and the importance of recent developer behavior.

Support for thesis comparison criteria For this thesis, the paper reinforces several evaluation criteria:

- **Stability:** Time-decayed metrics and multi-version reasoning reinforce the need for stability testing.
- **Interpretability:** The paper supports using simpler metrics like OnlyLast when possible, as they are easier to explain to stakeholders.
- **Resistance to gaming:** This paper is directly aligned with gaming resistance and the need to evaluate against common attacks (badmouthing, collusion, traitors).
- **Long-term suitability:** The proposed BlurredSquared metric emphasizes recent behavior but remains resilient over time, highlighting the importance of time-based metrics.
- **Feasibility:** The simulation-based approach reinforces the argument for feasible, auditable metrics that can be implemented on-chain in a minimal proof-of-concept.

How to reuse this paper in writing

- **Chapter 3:** Use the formal model and simulation approach as a methodological guide for evaluating metrics under attack scenarios.
- **Chapter 4:** Cite the attack scenarios and comparative findings (for example EigenTrust's weakness against selfish agents) to discuss limitations and trade-offs of chosen metrics.
- **Discussion and Conclusion:** Use BlurredSquared as an example of improving robustness by adjusting weighting over time.

8.4 Quick notes to remember

- Different reputation metrics resist different attacks; there is no universal best metric.
- Time decay is a useful mechanism to improve robustness.
- BlurredSquared is a strong candidate for long-term and attack-robust reputation metrics.
- The simulation framework demonstrates how to model reputation systems for testing.

8.5 Citation note

Schlosser, Voss, Brückner. "Comparing and Evaluating Metrics for Reputation Systems by Simulation." *2004 (confirm exact venue/year from the paper PDF).*

9 Paper 9: *A Survey of Blockchain: Techniques, Applications, and Challenges* (Gao et al., 2018)

9.1 Why this paper matters

What this paper is A broad, system-level survey that explains blockchain via a layered framework (data layer, network layer, application layer), then reviews major application domains, and finally summarizes core limitations and challenges (security and performance).

Why it is aligned with this thesis (evidence-based developer reputation and smart-contract computability)

- **Smart-contract feasibility criterion:** The paper explicitly positions smart contracts and automation as a major application direction and flags a key integration difficulty: connecting on-chain contract logic with off-chain and external services and real-world exchange of goods (an oracle and external-dependency style challenge). This directly supports this thesis requirement to judge which metrics are implementable on-chain without unsafe or fragile external assumptions.
- **Threat model grounding (gaming and manipulation):** The security section outlines attack classes rooted in consensus incentives (majority attack, selfish mining) and privacy leakage. While not about gaming metrics directly, it is very relevant for this thesis because it reminds us that any reputation or metric system placed on-chain inherits incentive attacks and must be designed under adversarial behavior assumptions.
- **Performance constraints:** The paper frames consensus as resource-wasteful and emphasizes scalability, availability, and throughput limits, including that blockchains often cannot compete with traditional databases under high workload. This supports this thesis stance: metric selection must respect on-chain cost constraints and throughput realities.

How this paper will be used in the thesis

- As a **foundational reference** to justify the smart-contract computability selection criterion.
- As a **design-constraints reference** to motivate computability rubric dimensions (computation cost, storage growth, need for external data, determinism, auditability).
- As a **threat-context reference** for the hard-to-game criterion (linking incentives and attack surfaces to metric-based reputation).

9.2 Short summary

This survey explains blockchain as a decentralized, immutable ledger where distributed nodes collectively affirm transaction provenance using cryptographic primitives and a consensus mechanism. It motivates blockchain as a response to weaknesses of centralized architectures (single point of failure, trust asymmetry between users and providers), especially under the growth of IoT, cloud and edge computing, and big data.

The authors propose a high-level blockchain framework with three layers:

- **Data layer:** blocks, transactions, hashes, Merkle trees, and digital signatures that provide integrity and traceability.
- **Network layer:** peer-to-peer participation, distribution of the ledger, and consensus protocols that decide which blocks are valid.

- **Application layer:** integration into domains such as IoT, big data, cloud and edge, identity management, cryptocurrency and markets, business solutions, smart contracts and automation, supply-chain traceability, medical informatics, and networking.

Technically, the paper covers transactions as payload; cryptographic hash pointers linking blocks; digital signatures for provenance; Merkle trees for compact integrity of many transactions; and block headers and bodies including previous-block hash, timestamp, Merkle root, and a consensus-related nonce. On the network side, it surveys consensus families such as Proof-of-Work, Proof-of-Stake, PBFT, and trusted-hardware approaches (for example PoET), describing their trade-offs and limitations.

Finally, the paper highlights two barriers to broad adoption: **security issues** and **performance issues**. Security issues include majority attacks, selfish mining (even with less than 50% power), anonymity and privacy risks due to public transactions and side channels (for example IP linkage and compromised third-party services), and abuse scenarios (including illicit activity and persistence of arbitrary content stored on-chain). Performance issues include scalability (ledger growth, distribution overhead, throughput limits due to block size and time restrictions) and availability and latency, noting benchmarking work that shows common platforms can lag far behind conventional databases under heavy workload. The conclusion positions blockchain as maturing but constrained: useful where auditability and decentralized trust matter, yet challenged by incentive attacks and system performance.

9.3 Detailed summary

1) Motivation and scope

- The survey frames blockchain as a response to trust and security pressures in distributed environments: more connected devices, larger data flows, and repeated breaches in centralized providers.
- It aims to: (i) introduce blockchain fundamentals and a structured framework; (ii) review application areas; (iii) discuss challenges (security and performance).

2) Layered framework: data, network, application

- **Data layer:** structures and cryptographic components that make records tamper-resistant and traceable.
- **Network layer:** P2P membership, ledger distribution, and consensus protocols that decide the valid chain state.
- **Application layer:** domain-specific systems that leverage the above (ledger, consensus, cryptographic validation, smart contracts).

3) Data layer (what is inside a blockchain and why it matters)

- **Transactions (data records):** basic payload, representing interactions (payments, data writes, and so on) at a particular time.
- **Hash and hash pointers:** cryptographic hashes provide integrity checks; hash pointers connect data blocks so modification breaks the chain.
- **Public-key cryptography and digital signatures:** provenance and authenticity, where issuers sign transaction material and receivers are identified via public keys.
- **Merkle tree:** transactions are hashed and aggregated; any change propagates upward, making the Merkle root a compact integrity commitment for the whole set.

- **Block structure:** block header typically includes version, previous-block hash, Merkle root, timestamp, difficulty target (nBits), nonce; block body includes transaction count and the transactions themselves.
- **Nonce and puzzle friendliness:** the nonce enables consensus puzzles (notably PoW) where solutions are hard to find but easy to verify.

4) Network layer (who participates, how agreement happens)

- **P2P network roles:** nodes can be full nodes (hold complete ledger copy) or miners and validators; ledger persistence emerges because many copies exist.
- **Consensus need:** without a trusted central party, nodes must agree on valid blocks and transactions; consensus determines which history is authoritative.

5) Consensus mechanisms summarized (and why this matters to on-chain feasibility)

- **PoW:** miners solve computational puzzles (nonce search); resilient but expensive; subject to forks and incentive-based attacks; difficulty cannot be too low (or excessive forks and double-spending risk).
- **PoS:** stake influences ability and difficulty; tries to reduce wasted work; still has incentive and pathology issues (for example nothing-at-stake), mitigated by deposits and penalties.
- **PBFT:** deterministic finality via multi-phase voting (pre-prepare, prepare, commit); communication cost is high, and naive implementations scale poorly.
- **Trusted hardware / PoET:** reduces energy costs by relying on trusted execution environments and attestation; shifts trust to hardware and trusted code base.
- **Other:** PoA (authorized validators with time windows) and PoB (destroy value to gain proposing rights).

6) Emergent applications (taxonomy-level value for this thesis) The survey groups blockchain use into several application domains. For this thesis, the main utility is:

- identifying **where** blockchain adds value (auditability, traceability, decentralized agreement),
- and where it struggles (resource constraints, high-throughput needs, privacy leakage, external dependencies).

Key domains:

- **IoT:** blockchain as trust layer; computational limits motivate offloading or private and permissioned designs.
- **Big data:** blockchain for trading, sharing, and traceability; often stores hashes and metadata on-chain and keeps bulk data off-chain.
- **Cloud and edge:** blockchain for contract management and removing reliance on trusted third parties; some designs use private chains plus cryptographic proofs.
- **Identity management:** blockchain-backed identity logs, access control, self-sovereign data control; often with off-chain storage plus on-chain commitments.
- **Cryptocurrency and markets:** variations in consensus and incentives illustrate that parameters and incentives shape security.

- **Smart contracts and automation:** appealing due to auditable transaction history plus programmable execution, but difficult to connect reliably to external services, external data, and real-world delivery.
- **Supply-chain traceability:** typically stores proofs and hashes on-chain; raw certificates and files may remain in conventional databases (hybrid architecture).
- **Medical informatics:** emphasizes privacy and access control; latency and availability constraints can become safety-critical.
- **Networking:** gateways and edge nodes as resource-rich anchors; policies and identities can be encoded via contracts.

7) Security concerns (directly relevant to hard-to-game criterion)

- **Majority attack:** controlling more than 50% of mining or validation can hijack history and introduce erroneous blocks.
- **Selfish mining:** attackers with less than 50% can still profit by withholding blocks and strategically releasing longer private chains, shifting incentives and potentially centralizing power.
- **Privacy leakage:** anonymous transactions can still leak identity via linkage attacks (for example IP association) or compromised third parties (exchanges, key managers).
- **Abuse and illicit use:** secure ledgers can also serve malicious actors; illicit activity measurement and arbitrary content insertion raise practical and legal risks.

8) Performance concerns (directly relevant to smart-contract computability)

- **Scalability:** chain size grows continually; distribution slows; public chains restrict block size and interval, lowering throughput.
- **Availability and latency:** confirmation time and commit reliability can be problematic; empirical benchmarking shows typical platforms can be far behind traditional databases at high load.
- **Implication for this thesis:** any metric implemented as a smart contract should avoid heavy on-chain computation, frequent writes, and large state growth.

9) Closing perspective The paper concludes that blockchain is maturing and useful for decentralized trust and auditability, but adoption is constrained by incentive attacks and performance limits; designers must select techniques appropriate to target business logic and workload.

9.4 How I will reuse it in the thesis

D1. What this paper gives that can be reused directly

- **Justification for smart-contract computability criterion:** The survey emphasizes that consensus is costly and that scalability and availability remain limiting. This supports a thesis argument that a metric is only a good candidate for an on-chain reputation proof-of-concept if it can be computed and updated under tight cost and throughput constraints.
- **A clean conceptual framework (layers) to structure PoC discussion:** The PoC can be described with the same layered thinking:

1. *Data layer:* what is stored (metric values, commits, developer identifiers, project identifiers, hashes).
2. *Network layer:* assumptions about consensus and permissioning (public vs consortium vs private).
3. *Application layer:* how reputation queries and updates happen, and what incentives exist.

- **Explicit acknowledgement of smart-contract integration difficulty:** The survey flags the challenge of tying smart contracts to external services and real-world events. For this project, this means: if a metric needs off-chain facts (for example who authored which commits, issue-tracker activity, review quality), the PoC should either:

  1. avoid those dependencies by selecting an on-chain-native metric, or
  2. represent them as signed attestations posted on-chain (with explicit trust and oracle assumptions).

D2. Implications for metric-selection criteria Supervisors require stability, interpretability, gaming resistance, long-term indicator quality, and smart-contract feasibility. This paper strengthens the last criterion and gives supporting logic for the others:

- **Gaming resistance:** In blockchains, adversaries exploit incentives (selfish mining) and majority power. By analogy, a metric-based reputation system will also be attacked through incentives: developers may optimize for the metric rather than quality, and actors may try to manipulate inputs. So gaming-resistance criterion should explicitly define:

  1. attack surface (what inputs can be faked or cheaply manipulated),
  2. cost to manipulate (how expensive it is to generate fake events),
  3. detectability and auditability (whether manipulation leaves on-chain traces).

- **Interpretability:** The survey's explanation of how data integrity is derived (hash pointers, Merkle roots, signatures) supports a narrative that on-chain metrics can be auditable. Interpretability becomes: can a stakeholder understand what a metric means, and can they verify provenance of input data?

- **Stability across versions and projects:** While not its central focus, the paper's emphasis on immutable histories suggests a benefit for longitudinal tracking. If metric snapshots can be defined per release and commit, temporal stability can be studied, but feasibility still depends on storage and update-frequency constraints.

D3. Concrete smart-contract feasibility constraints to operationalize The survey repeatedly highlights cost and performance limitations plus the smart-contract external-data challenge. Translating that into a feasibility checklist:

1. **On-chain compute cost:** Can the metric be computed with a small number of operations per update?
2. **On-chain storage growth:** Does the metric require storing large histories, or can only aggregates be stored (for example counts, sums, rolling windows)?
3. **Update frequency:** Does the metric need an update per commit, PR, or issue event, or can it be computed per release or per time window?
4. **Deterministic inputs:** Are inputs purely on-chain facts (transactions and events), or is off-chain data required (introducing oracles and trust assumptions)?

5. **Auditability:** Can any third party recompute and verify the metric from stored commitments, hashes, and public events?

6. **Adversarial robustness:** If the metric is public, how easy is it to produce cheap fake events to inflate it?

D4. How this paper narrows PoC design space

- Prefer metrics that are **aggregate-based** and **incrementally updatable** (counts, sums, bounded statistics) rather than metrics requiring heavy static analysis on-chain.
- If code metrics are still used (complexity, churn), then computation will likely happen off-chain and the chain stores **attested results plus hashes** of analyzed artifacts, with a clearly stated trust model (who attests and how disputes are handled).
- The application survey suggests many real systems are **hybrid**: store hashes and metadata on-chain, store heavy data off-chain. This is a defensible architecture for this metric proof-of-concept if justified explicitly.

D5. Ready-to-use writing points

- **Motivating computability:** Blockchain offers auditability and decentralized agreement but faces security and performance limitations. So not every metric is suitable for smart-contract implementation. Metrics requiring high computation, high-frequency updates, or large state growth are weak proof-of-concept candidates; metrics with incremental updates and small state are stronger candidates.
- **Motivating oracle caution:** While smart contracts allow automated execution, tying on-chain logic to external services and real-world events is difficult and introduces added trust assumptions. Therefore, feasibility criterion should explicitly penalize metrics that require unverifiable external inputs.

9.5 Quick notes to remember

- This paper gives a strong foundation for the smart-contract computability criterion.
- Security and incentive attacks in blockchain map naturally to metric-gaming risks in reputation systems.
- Performance constraints imply preference for simple, incremental, low-state on-chain metrics.
- Hybrid architecture (off-chain heavy data, on-chain proofs and hashes) is often the practical design choice.

9.6 Citation note

Gao, Hatcher, Yu. "A Survey of Blockchain: Techniques, Applications, and Challenges." *IEEE, 2018.*

10 Paper 10: *Assessing Vulnerability in Smart Contracts: The Role of Code Complexity Metrics in Security Analysis* (Tehrani, 2026)

10.1 Why this paper matters

A) Alignment & Fit to Thesis

Thesis fit (why this paper is in the core set) This paper is **highly aligned** with this thesis because it connects three elements:

1. **Smart contracts are high-stakes and hard to patch** (immutability plus money at risk), which motivates strong pre-deployment assessment.
2. **Metrics as evidence-based signals:** it evaluates whether software metrics (here, complexity metrics) can function as diagnostic signals for security risk.
3. **Multi-metric reality:** it empirically supports the point that single metrics are weak alone but become useful collectively. This maps directly to the thesis agreement: compare multiple metrics; implement one metric as proof-of-concept.

Direct contributions to thesis research questions

- **Stability and robustness argument:** Individual metric-vulnerability associations are weak, and redundancy exists among metrics. This strengthens the plan to evaluate stability and avoid over-claiming from one metric.
- **Interpretability:** The metrics are mostly classical (LOC-like size metrics, cyclomatic-complexity style metrics, coupling and inheritance analogs). These are explainable to non-experts, which is good for reputation interpretability.
- **Gaming and manipulation:** Findings imply some metrics can be superficially altered (for example comment lines) without improving real quality. This supports gaming resistance as a formal comparison dimension.
- **Smart-contract feasibility:** Complexity metrics are easy to compute with static-analysis tools off-chain, but expensive to compute on-chain. This justifies a hybrid proof-of-concept design (on-chain storage and verification, off-chain computation).

Where this fits in the final thesis structure

- **Background:** why smart-contract quality and security are unique (immutability, high-value targets).
- **Related work:** metrics and vulnerability prediction (traditional languages vs Solidity).
- **Methodology inspiration:** statistical testing choices (Spearman, Mann-Whitney, effect size).
- **Core narrative evidence:** single metric weak; multiple metrics better; redundancy matters; complexity indicates risk but is not a cause.

10.2 Short summary

B) Short Summary

This paper studies whether **code complexity metrics** can help identify **vulnerable Solidity smart contracts**. The motivation is that smart contracts are immutable after deployment, execute autonomously, and often manage funds, so vulnerabilities can cause catastrophic losses. The author treats complexity as a quantifiable, objective signal that has historically correlated with defects and vulnerabilities in traditional software, and asks whether a similar relationship holds in Solidity.

The study extracts **21 complexity metrics** from Solidity source code using **Solmet**, a static-analysis tool for smart-contract metrics. The metrics cover size (SLOC, LLOC), documentation (CLOC), functional structure (number of functions), control-flow complexity (McCabe-style complexity, nesting), coupling and invocations (fan-out / number of outgoing invocations), and OO-style inheritance and coupling proxies (DIT, ancestors and descendants, coupling between objects). The dataset comes from prior work that crawled verified contracts (Etherscan), removed duplicates, deployed them locally, and labeled them for **eight vulnerability types** (for example reentrancy, integer overflow and underflow, timestamp and block dependency, dangerous delegatecall, unchecked external call).

The analysis answers four research questions: (1) whether metrics are redundant (correlated with each other), (2) whether each metric correlates with vulnerability presence, (3) whether metric distributions differ between vulnerable and neutral code, and (4) how strong the higher-complexity to more-vulnerability pattern is.

Results show that **many metrics are highly correlated** (redundant feature sets), and that **each individual metric correlates only weakly** with vulnerabilities. However, most metrics **strongly discriminate** vulnerable versus neutral contracts when comparing distributions (with statistically significant differences and often large effect sizes). In general, vulnerable contracts show higher mean complexity, but there are notable exceptions: **comment lines**, **descendants**, and **average parameters** tend to be lower in vulnerable code. The conclusion is that complexity is an **indicator, not a direct cause** of vulnerability; still, complexity metrics are valuable as **complementary signals**, especially when used together, and can support automated risk scoring and audit prioritization.

10.3 Detailed summary

C) Detailed Summary

C1. Motivation: why smart-contract security makes metrics more critical The paper frames smart contracts as a special software-engineering case where:

- Code is **immutable** after deployment (upgrades are difficult or indirect), so post-release patching is constrained.
- Contracts can **hold funds** and execute automatically, raising attacker incentives.
- High-profile incidents show real financial damage and motivate systematic pre-deployment assessment.

From this, the paper motivates complexity metrics as a scalable way to provide objective risk signals that can be computed automatically.

C2. Research questions (what exactly is tested) The work is structured around four RQs:

1. RQ1 (redundancy): Are complexity metrics correlated with each other?

2. RQ2 (individual association): Does each metric correlate with vulnerability presence?
3. RQ3 (discriminative ability): Do vulnerable and neutral code differ in metric distributions?
4. RQ4 (direction and magnitude): Are metrics systematically higher (or lower) in vulnerable code?

This is important for this thesis because it separates association (RQ2) from practical discriminability (RQ3 and RQ4), which prevents over-interpretation.

C3. Why complexity is investigated The argument is that complexity increases cognitive load and maintenance difficulty, creating more opportunities for errors; security experts have long warned that complexity indirectly increases attack surface. The paper also emphasizes that complexity should be treated as an **indicator** rather than a causal mechanism.

C4. Solidity is not traditional software: implications for metric behavior The paper highlights Solidity and EVM properties that may change metric behavior:

- Solidity's smaller ecosystem (types and libraries) and deterministic constraints.
- Domain-specific vulnerability classes (reentrancy, transaction ordering, and others).
- Gas costs: developers may limit control structures to reduce gas, affecting classic complexity measures.
- Persistent state and modifiers, payable functions, and language-specific data types.

This supports a key thesis theme: metric results cannot be transferred from traditional languages to smart contracts without validation.

C5. Metric set (21 metrics) and what they represent Metrics are extracted per contract with Solmet and cluster into families:

- **Size:** SLOC (all lines), LLOC (logical/executable lines), and related size effects.
- **Documentation:** CLOC (comment lines), enabling comment-ratio reasoning when combined with LLOC.
- **Functional structure:** NF (number of functions), NUMPAR (parameters), NOS (statements).
- **Control-flow complexity:** WMC and Avg.McCC (McCabe-like complexity), NL/NLE and their averages (nesting depth).
- **Coupling and invocations:** NOI and Avg.NOI (fan-out / outgoing invocations).
- **Inheritance and coupling proxies:** DIT (depth of inheritance), NOA/NOD (ancestors/descendants), CBO (coupling between objects).
- **Contract attribute vocabulary:** NA (number of attributes, modifiers, and keywords used).

For this thesis, this taxonomy is reusable for interpretability and for explaining what each metric actually measures.

C6. Dataset and labeling (ground truth) The dataset is sourced from prior work (IR-Fuzz) that collected verified contracts and labeled them for eight vulnerability types (overflow/underflow, strict equality hazards, reentrancy, timestamp dependency, block-number dependency, delegatecall hazards, ether frozen, unchecked external call). Labeling uses a two-stage process: pattern-based initial identification and manual verification to reduce labeling cost.

An important nuance is heavy class imbalance (few vulnerable contracts versus many neutral). This affects interpretation: statistical significance may not imply strong practical prediction power, and weak correlation is not surprising.

C7. Statistical methods (why these tests, and what they answer) The paper uses a coherent chain:

- **Spearman correlation** for monotonic relationships:
  1. metric-metric correlations (RQ1) to detect redundancy;
  2. metric-label correlations (RQ2) to test individual association with vulnerability presence.
- **Mann-Whitney U test** (non-parametric) to compare vulnerable versus neutral metric distributions (RQ3).
- **Cliff's Delta** as an effect size to quantify how often vulnerable values exceed neutral values (practical magnitude).
- **Confidence intervals** to visualize group differences (RQ4), emphasizing direction and exceptions.

This is a strong methodological reference for defining stability and difference across versions or projects.

C8. Results - RQ1 (redundancy among metrics) The paper reports multiple strong metric-metric correlations, implying redundancy:

- SLOC strongly relates to NOI, NOS, WMC, NF, and LLOC.
- NF and WMC, although conceptually different, correlate strongly (plausibly due to gas-conscious coding reducing variation in control structures).
- Nesting metrics (NL, NLE, Avg.NL, Avg.NLE) can become effectively interchangeable in Solidity due to limited use of deep if and loop structures.
- Inheritance-related metrics (NOA and DIT) show similarity.
- NOI and Avg.NOI can be equal or near-equal when invocation patterns are limited.

Thesis relevance: in a multi-metric reputation model, redundancy must be controlled to avoid double-counting size or structure.

C9. Results - RQ2 (each metric vs vulnerability label) All metrics show statistically significant correlations with vulnerability presence, but **the magnitude is weak for every metric**. This is crucial:

- It rejects a one-metric-to-rule-them-all approach.
- It implies single-metric reputation systems are likely fragile and misleading.
- It supports the thesis stance: compare multiple metrics, then implement only one for proof-of-concept due to feasibility constraints.

The paper explicitly contrasts this with traditional-language studies where stronger correlations were observed.

C10. Results - RQ3/RQ4 (discriminative power and direction) Despite weak individual correlation, distributional tests show:

- All metrics show statistically significant differences between vulnerable and neutral groups (Mann-Whitney U).
- Many metrics have **large effect sizes** (Cliff's Delta), meaning vulnerable contracts tend to score higher.
- Exceptions exist: **CLOC**, **NOD**, and **Avg.NUMPAR** trend lower in vulnerable code (negative Cliff's Delta with small magnitude).

This gives a nuanced picture: complexity is generally higher in vulnerable code overall, but specific signals (documentation volume, extensibility proxies, parameter averages) can behave differently in Solidity.

C11. Discussion: practical integration and best practices The paper proposes concrete uses:

- **Feature optimization:** drop redundant metrics to reduce overhead and increase interpretability.
- **Risk assessment:** treat metrics as complementary features in a risk score (some metrics like coupling/invocations may be weighted higher).
- **Audit prioritization:** flag high-complexity contracts or regions for deeper review.
- **Developer guidance:** keep contracts simple, reduce deep nesting, avoid overly complex inheritance, and note that documentation may be protective (CLOC higher in neutral contracts).

C12. Threats to validity (why interpretation must stay cautious) The paper is explicit about limitations:

- Dataset and labeling choices can shift results; different datasets may produce different outcomes.
- Solidity version distribution may matter; older syntax differs.
- Tool validity: Solmet was introduced earlier; dataset includes newer contracts, but the paper argues metrics are traditional enough to remain meaningful.
- Domain confounding: higher complexity may reflect inherent problem complexity (for example DeFi vs simple tokens), not bad style, motivating stratification by contract type.

This directly informs thesis design: stratify or control for project and contract domain when possible.

C13. Main conclusion (message to reuse) The paper concludes:

1. Some metrics are redundant and strongly correlated.
2. Individual metrics alone are weak predictors (low correlation).
3. Collectively, metrics can discriminate vulnerable versus neutral code strongly.
4. Most metrics are higher in vulnerable code, with a few meaningful exceptions.

It also stresses that complexity is associated with vulnerability but is not necessarily causal.

10.4 How I will reuse it in the thesis

D) Thesis-Oriented Notes

D1. Evidence for metric-selection philosophy This paper is very useful for two thesis claims:

1. **Single-metric caution:** one metric is usually insufficient to explain or predict vulnerability risk; correlation magnitudes are weak.
2. **Multi-metric advantage:** even with weak individual correlations, metrics can still show strong discriminative power collectively and support prioritization decisions.

This aligns with supervisory feedback that multiple metrics are usually needed.

D2. How it changes definition of a good reputation metric For an evidence-based developer reputation system, do not confuse:

- **association** (weak correlation with a label), and
- **utility** (ability to discriminate groups, rank risk, and support decisions).

A metric can still be operationally useful with low correlation if combined and interpreted correctly. So comparison should separate:

1. standalone predictive signal;
2. complementary signal in a metric bundle.

D3. Redundancy as a first-class selection criterion This paper strongly supports adding redundancy/collinearity as a criterion:

- If SLOC correlates strongly with multiple metrics, a formula that sums all of them unintentionally overweights size.
- The comparison should explicitly record whether a metric adds unique information or mostly repackages size.

This is a defensible reason to prefer compact metric sets and justify final metric selection.

D4. Smart-contract computability implications Key translation into the smart-contract feasibility rubric:

- Metrics are **easy to extract via static analysis off-chain**, but full static analysis **on-chain** is unrealistic because of gas and determinism constraints.
- A realistic proof-of-concept pattern is:
  1. compute metric off-chain from a verified source artifact (for example commit hash / release tarball hash),
  2. post **attested metric value plus artifact hash** on-chain,
  3. optionally allow disputes / re-verification mechanisms (depending on project scope).
- For a minimal proof-of-concept, choose a metric that can be incrementally updated from events (for example counts, simple aggregates) or posted as periodic snapshots.

D5. Candidate metrics suggested for implementation shortlist If one metric must be implemented as proof-of-concept, this paper suggests two directions.

(1) Simple, explainable, low-cost metrics (good for feasibility):

- SLOC/LLOC family (size): explainable, but gameable through verbosity and not strongly causal.
- NF / NOS: structural counts, feasible to store and update, but can still correlate with size.

(2) Slightly richer metrics (still explainable) that may capture more risk than pure size:

- Control-flow complexity (Avg.McCC, nesting): linked to reasoning difficulty, but computing from source needs parsing.
- Coupling/invocation metrics (NOI/CBO): may represent dependency surface; more meaningful but still off-chain computed.

The thesis should not present a best metric, but a feasible metric with defensible meaning.

D6. Gaming-resistance insights Exception metrics matter for gaming analysis:

- **CLOC higher in neutral code** suggests documentation may correlate with safety; but as a reputation metric, comment count is highly gameable.
- If comment-related metrics are included, protections should be justified (for example normalize, cap contributions, or treat as secondary support signal).
- **Size metrics** can be manipulated by refactoring style (splitting/merging functions, formatting changes). So size metrics should score low on hard-to-game.

D7. Method reuse for Step 3 evaluation This structure can be reused for evaluating chosen metrics on OSS projects:

1. Compute metric values across versions/releases.
2. Use non-parametric tests for distribution shifts (metrics are rarely normal).
3. Report both significance and effect sizes (avoid p-value-only reporting).
4. Explicitly test redundancy if multiple metrics are included.

D8. What not to over-claim This paper is careful: complexity correlates with vulnerability but is not causal. The same discipline should be preserved:

- Do not claim metric causes quality/vulnerability.
- Claim metric is an indicator/proxy with observed association and practical discriminative utility under conditions.
- Acknowledge confounders: domain complexity, contract type, and dataset bias.

10.5 Quick notes to remember

- Individual complexity metrics show weak correlation with vulnerability labels.
- Multi-metric views can still discriminate vulnerable vs neutral code effectively.
- Redundancy among metrics is substantial and must be controlled.
- Complexity is a useful risk indicator, but not a causal proof.
- Off-chain computation plus on-chain attestation is the realistic smart-contract metric pattern.

10.6 Citation note

Masoud Jamshidiyan Tehrani. "Assessing Vulnerability in Smart Contracts: The Role of Code Complexity Metrics in Security Analysis." *arXiv v4, January 2026.*

11 Paper 11: *Smart contracts software metrics: A first study* (Tonelli et al., 2023)

11.1 Why this paper matters

A) Alignment & Fit to thesis

What this paper is (in one sentence) A large-scale empirical and statistical study on about 85K Solidity smart contracts that computes internal smart-contract metrics and external/interface metrics (ABI and bytecode), then compares their distributions to those known in traditional software.

Why it is core to this thesis (mapped to agreed criteria) This paper directly supports the smart-contract feasibility criterion and strengthens the comparison framework:

1. **Smart-contract feasibility (computability plus constraints):** The paper is built around blockchain constraints: limited size, per-operation execution cost (gas), immutability after deployment, no floating point, and no true randomness. These are exactly the constraints needed in the feasibility rubric and proof-of-concept design.

2. **Interpretability:** Most metrics are plain and explainable counts (LOC, number of functions, events, mappings, modifiers, payable functions, cyclomatic complexity, and others). This is useful for evidence-based reputation because stakeholders can understand these signals.

3. **Stability (what stability should mean in this domain):** The study shows smart-contract metric ranges are more restricted than traditional software, and distributions often have upper cut-offs driven by deployment constraints. This shapes stability definition: metrics may be naturally bounded and less volatile, but may also saturate and lose discriminative power.

4. **Gaming and manipulation:** The paper does not directly measure gaming, but it gives empirical grounding: when metrics are bounded and developers face gas and size constraints, developers may optimize for deployment feasibility in ways that reshape metric values (for example fewer loops/branches). That creates an incentive channel and a gaming surface for reputation metrics.

5. **Long-term indicator suitability:** The paper reports changes across pragma/compiler versions, including shifts in LOC distributions and a trend reversal after Solidity 0.8. This is directly relevant to long-term indicator suitability: metrics used for reputation should account for language/toolchain evolution and ecosystem shifts.

How this paper will be used in the thesis

- As **primary evidence** that smart-contract metrics behave differently (restricted ranges, cut-offs, different statement-usage densities), supporting a dedicated feasibility rubric and careful metric choice.

- As a **source of candidate metrics** that are realistic to compute off-chain and realistic to store/verify on-chain in a proof-of-concept, especially simple counts and ABI/interface exposure proxies.

- As **methodological inspiration** for distributional analysis (CCDF, power-law vs log-normal fitting, bootstrap confidence intervals), reusable for stability analysis across versions/projects.

11.2 Short summary

B) Short Summary

This paper argues that smart contracts are software artifacts developed under constraints that differ sharply from traditional systems. Because smart contracts are deployed as bytecode on-chain, they should remain lightweight in code size, and execution has a per-operation gas cost. Once deployed, contracts are effectively immutable (hard to update). Further, contracts must execute deterministically across many nodes, which excludes typical constructs such as floating point arithmetic and true randomness. The authors hypothesize that these constraints should be visible in statistical properties of smart-contract metrics, and may motivate smart-contract-specific metrics.

To test this, the authors analyze a large dataset of about 85K Solidity contracts collected from Etherscan and Smart Corpus. They extract source code, ABI, and bytecode for each contract, parse the source into an AST (using PASO), and compute metrics including: total lines, blank lines, comment lines, LOC, number of contracts per address, functions, payable functions, events, mappings to addresses, modifiers, interfaces/libraries, cyclomatic complexity, and ABI/bytecode sizes. They compute descriptive statistics and then analyze full distributions instead of relying only on histograms.

A central result is that smart-contract metrics tend to have **restricted ranges** compared with traditional software. Many metrics exhibit **upper cut-offs** consistent with on-chain size/cost constraints. As a consequence, classic fat-tail/power-law patterns often seen in traditional software appear only partially or only in limited regions. LOC is a notable exception: it behaves more like traditional software, is well described by a log-normal distribution, and shows an approximate truncated power-law tail. ABI and bytecode metrics display bell-shaped behavior around central values, suggesting many contracts have a typical level of external interface exposure.

The paper also studies statement usage and finds that iteration and conditional statements per LOC are far lower than in languages such as Java/C/Python, consistent with developers avoiding expensive branching/loops and trying to keep contracts cheap and understandable. Finally, it reports evolution over compiler/pragma versions: distributions shift toward larger LOC ranges up to Solidity 0.7, with trend reversal after Solidity 0.8, plausibly linked to compiler-level safety changes and reduced need for external libraries.

11.3 Detailed summary

C) Detailed Summary

C1. Domain constraints (paper axioms) The paper lists constraints that are structural properties of the platform:

- **Lightweight code:** size limited by blockchain constraints and deployment/mining cost.
- **Costed execution:** each operation burns gas; expensive code is economically discouraged.
- **Immutability:** bytecode is inserted into append-only ledger; updates are difficult.
- **Determinism constraints:** floating point and true randomness are disallowed/unsafe because all nodes must reach identical state.

This is central for deriving feasibility dimensions (cost, determinism, update constraints).

C2. Operational Ethereum model (why ABI/bytecode matter) The authors model contracts as accounts with code and state, executed deterministically. Invocation includes payment and input data; deployment stores bytecode on-chain and initializes state with constructor

logic. The gas mechanism is explicit: users set gasLimit and gasPrice; if execution exceeds gasLimit, state rolls back but gas is still paid. This discourages resource-heavy execution.

Thesis implication: compute-on-chain is expensive, so store-minimal-and-verify designs are attractive for proof-of-concept metrics.

C3. Metric design rationale The paper notes that many OO-style metrics adapt easily (LOC, comments, number of functions, cyclomatic complexity), but contract-to-contract communication differs due to transaction/message semantics. So the study includes both:

- classic source metrics (size/structure/complexity),
- smart-contract-specific indicators (payable functions, mapping-to-address usage, events, modifiers, ABI size as interface/exposure).

C4. Dataset and extraction pipeline

- Sources: Etherscan (source, ABI, bytecode) and Smart Corpus.
- Stored data: address, source, ABI, bytecode; then parse and compute metrics.
- Replication patterns exist (same/near-same code at different addresses), but authors argue bias is limited because replicated-code cases are relatively few.

C5. Internal vs external metrics (important split) Key conceptual distinction:

- **SCIM (internal metrics):** derived from Solidity source (LOC, functions, cyclomatic, etc.).
- **SCEM / interface side:** ABI and bytecode metrics, reflecting exposure and deployed-artifact constraints.

The paper also notes that many address-related measures are time-varying (popularity, call counts, ether balance), so they require caution if correlated with code metrics.

C6. Descriptive statistics (what a typical contract looks like) Reported centrality/dispersion patterns are right-skewed: mean is greater than median for most metrics, and maxima can be one to two orders of magnitude above central values, but not the extreme multi-order ranges typical of unbounded traditional software metrics.

Selected anchor values:

- `LOC`: mean 306.63, median 167, max 14,151; 90th percentile 663.
- `functions`: mean 44.96, median 28, max 1,256; 90th percentile 95.
- `cyclomatic`: mean 66.50, median 36, max 2,318; 90th percentile 146.
- `bytecode`: mean 12,483, median 9,606, max 49,152 bytes.
- `abiStringLength`: mean 4,644, median 3,886, max 48,274.

Interpretation: many metrics show meaningful variation but still appear bounded enough to avoid extreme tails.

C7. Statement-level statistics (style shaped by cost constraints) The paper reports much lower loops/branches per LOC compared with common general-purpose languages:

- Typical contract has about 10 `if` statements, about 5 `emit` statements, and around 1.5 iteration statements (for/while combined).
- Iteration statements per LOC (0.005) and conditional statements per LOC (0.033) are far lower than Java/C/Python references.
- `emit` is highlighted as a smart-contract-specific statement relevant for dApp event emission.

Thesis relevance: developer behavior is constrained by execution cost and readability expectations, which changes meaning, stability, and gameability of complexity metrics.

C8. Distribution-analysis strategy The authors treat histograms as fragile due to binning/tail noise and move to:

- empirical CCDF (log-log) for tail visualization,
- fitting of power-law tails (with $x_{\min}$) and log-normal distributions,
- goodness measures (KS values) and bootstrap confidence intervals.

C9. Key distribution findings Headline result: **smart-contract metric distributions are generally more restricted** than traditional software distributions.

- Many metrics do not sustain a clear power-law tail; even when a power-law-like segment appears, it is often narrow.
- Some metrics show bell-shaped behavior (especially ABI and bytecode), implying a typical interface/exposure scale and strong impact of size constraints.

C10. LOC behaves differently from most metrics LOC is closest to traditional software behavior:

- LOC is well described by log-normal across bulk and tail, and shows an approximate power-law regime in the upper tail.

Thesis relevance: LOC remains broadly comparable across paradigms, though still truncated by on-chain constraints.

C11. Evolution by pragma/compiler version The study groups by pragma version (excluding cases with missing pragma) and reports:

- rightward shifts in peak distributions for functions/LOC/ABI from 0.4.* to 0.5.*,
- broadening followed by reversal after Solidity 0.8,
- growth in LOC/bytecode averages up to v0.7 then reduction at v0.8.

The proposed explanation is language/compiler evolution and safety features (notably 0.8 overflow checks) reducing dependence on external libraries.

C12. Code-size limit and diamond pattern The paper references the 24,576-byte contract code-size limit (introduced to reduce DoS risks), and discusses architecture patterns splitting logic across addresses (for example diamond/facets) to bypass single-contract limits.

Thesis implication: per-contract metrics can be misleading when logic is intentionally distributed across contracts. Reputation metrics should either:

1. evaluate a system as a set of contracts, or
2. detect modularization patterns and adjust interpretation.

C13. Final paper conclusions The paper concludes:

- ABI exposure is relatively similar across contracts (few extreme outliers; bell-shaped behavior),
- many metrics are limited by resource constraints and therefore do not show ubiquitous fat tails,
- LOC is closest to traditional software distribution behavior (log-normal plus truncated tail).

11.4 How I will reuse it in the thesis

D) Thesis-oriented notes

D1. Empirical backbone of smart-contract feasibility criterion This paper provides the strongest empirical chain for feasibility:

1. On-chain resources are limited (size cap, gas cost).
2. Developers respond by changing style (fewer loops/branches, smaller/simpler logic).
3. Therefore metric values/distributions are structurally shaped by platform constraints.

This justifies a dedicated feasibility rubric instead of blindly importing classic software-metric assumptions.

D2. Restricted ranges change metric usefulness If a metric is narrow and upper-bounded:

- **Pros:** potentially more stable, less sensitive to small refactorings.
- **Cons:** possible saturation (many contracts near typical value), reducing discrimination among developers/projects.

So stability should be paired with discriminative power/resolution in the comparison framework.

D3. ABI and bytecode as reputation-relevant signals Bell-shaped ABI/bytecode distributions suggest a typical interface exposure scale:

- ABI length / ABI string length can proxy public attack surface/interface complexity.
- Bytecode size is directly tied to deployment feasibility/cost.

For reputation, these can be treated as quality-risk indicators, but interpreted carefully because design patterns (for example facet modularization) can distort per-contract readings.

D4. Critical validity warning: diamond pattern and unit of analysis Facet-based decomposition can reduce per-contract LOC/complexity while shifting complexity to system level. This informs thesis design: smart-contract computable metrics should still map to a meaningful unit (single contract vs system of contracts).

D5. Methodology reuse for stability across versions Reusable choices from this paper:

- CCDF and model fitting instead of histogram-only analysis.
- Bootstrap confidence intervals for parameter estimates.
- Distribution comparison across compiler/pragma eras to detect ecosystem drift.

For stability definition, this supports:

1. within-project version-to-version variation,
2. distribution-shift checks across releases,
3. sensitivity to small diffs.

D6. PoC metric shortlist suggested by this paper Feasibility-compatible candidates for minimal on-chain proof-of-concept (store/verify, not heavy compute):

- **Simple counts:** LOC, number of functions, number of events, number of mappings-to-address, number of payable functions, number of modifiers.
- **Interface exposure:** ABI length / ABI string length as attack-surface proxy.

  Why strong for PoC:

- easy to explain,
- likely stable under small changes if normalized,
- computation off-chain, on-chain storage as values plus hash commitments.

  Why risky:

- LOC/count metrics are gameable (verbosity/refactoring),
- ABI metrics can be shifted by modularization patterns.

So these are practical PoC metrics, but should score lower on hard-to-game.

D7. Two thesis-ready argument blocks enabled by this paper

- **Why feasibility must be first-class:** Smart contracts run under strict size and execution-cost constraints; evidence across about 85K contracts shows many metric distributions are truncated and limited. So not every software metric remains meaningful/implementable on-chain; feasibility and interpretability should be first-class selection criteria.
- **Why ecosystem drift matters:** Metric distributions shift across compiler/pragma versions, with growth in size/structure metrics up to Solidity 0.7 and reversal near 0.8. Long-term reputation indicators should account for toolchain evolution to avoid unfair cross-era comparisons.

D8. Final caution for thesis claims This paper reinforces disciplined interpretation:

- use metrics as indicators/proxies, not causal proof,
- control for platform constraints and architectural patterns,
- separate stability from discriminative power.

11.5 Quick notes to remember

- Smart-contract metric ranges are often restricted and truncated by platform constraints.
- LOC is the closest metric to traditional software distribution behavior.
- ABI and bytecode provide practical interface/exposure signals.
- Compiler/version drift changes metric distributions and should be modeled.
- For PoC, simple counts and ABI-size proxies are feasible but partly gameable.

11.6 Citation note

Tonelli, Pierro, Ortu, Destefanis. "Smart contracts software metrics: A first study." *PLOS ONE, 2023.*

12 Final Synthesis and Closing Notes

This literature review now includes 11 papers and provides a complete evidence base for the thesis methodology. The main conclusion across papers is consistent: no single metric is enough for robust quality or reputation assessment, especially in smart-contract ecosystems with hard platform constraints and evolving toolchains.

The final metric-comparison framework for the thesis should keep five core dimensions:

- stability across versions/projects,
- interpretability for stakeholders,
- resistance to gaming/manipulation,
- long-term suitability under ecosystem drift,
- smart-contract feasibility (cost, storage, determinism, auditability, oracle dependence).

For implementation scope, the most defensible proof-of-concept pattern is hybrid: compute metrics off-chain from verified artifacts, then store attested metric outputs and artifact hashes on-chain for auditability and reproducibility.